

High Performance Computing 2026S Assignment 2

Issue date: 2026-05-19

Due date: 2026-06-09

No extensions.

Please do not forget to register with a group on TUWEL! Otherwise, you will not be able to submit your answers.

What to Hand in via TUWEL?

1. A **PDF file** containing your answers, using the provided report template.
2. Your **source codes** in three subfolders named `exercise1`, `exercise2`, and `exercise3` in `assignment2.zip`, compiled and executed with the requested commands in each exercise. **Do not** submit RAR, Word, or other formats.
3. Your **raw code outputs** and **nsys outputs** in **.nsys-rep** format for the requested exercises as `data.zip` containing three subfolders named `exercise1`, `exercise2`, and `exercise3`.

Rules

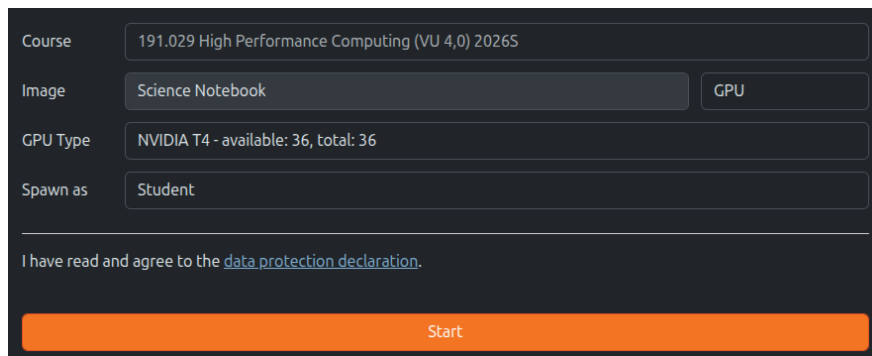
- Ensure that the following information is provided in the PDF document:
 - names of authors
 - date
 - course and assignment name
 - TUWEL group size and number
 - * Example: If you are registered for “3 Person Group 12” on TUWEL, then your group size is 3 and your group number is 12.
 - Use a spell checker. Check your grammar.
 - Label your figures and tables appropriately, e.g., “Table 1: Summary” is not an appropriate description.
 - Reference each figure and table in the text. Ensure that each figure/table is indeed described in the text. Only stating that “Table 1 contains a summary.” is not considered appropriate. Please explain what exactly is summarized in a table.
-

General Description

To complete this assignment, you will need access to a GPU machine through the JupyterHub environment. Go to TUWEL, select JupyterHub, choose the High Performance Computing course, and select a **Tesla T4** GPU from the server options (see Figure 1). Once logged in, open a **Terminal** from the JupyterLab launcher.

By default, the CUDA compiler is available at `/usr/local/cuda/bin/nvcc` and the NVIDIA Nsight Systems (nsys) profiler at `/usr/local/cuda/bin/nsys`.

You can install `nsys-ui` (suggested v2026.2.1) on your PC to visualize the profiler's logs. Older versions may be incompatible or omit information required for the exercises.



The image shows a JupyterHub configuration interface with a dark theme. It contains several input fields and a button. The 'Course' field is set to '191.029 High Performance Computing (VU 4,0) 2026S'. The 'Image' field has a dropdown menu showing 'Science Notebook' and a 'GPU' button next to it. The 'GPU Type' field shows 'NVIDIA T4 - available: 36, total: 36'. The 'Spawn as' field is set to 'Student'. Below these fields is a line of text: 'I have read and agree to the [data protection declaration](#)'. At the bottom is a large orange button labeled 'Start'.

Course	191.029 High Performance Computing (VU 4,0) 2026S
Image	Science Notebook GPU
GPU Type	NVIDIA T4 - available: 36, total: 36
Spawn as	Student

I have read and agree to the [data protection declaration](#).

Start

Figure 1: Jupyter hub setup required for the exercise.

Exercise 1 (12 [5+3+4] points): Implementing CUDA Streams

You are given a GPU pipeline in `exercise1/exercise1.cu` that applies a transformation followed by a prefix sum over a large array of floats. The host code is already implemented. The code is structured as following:

- **CPU reference** — `cpu_pipeline()`: already implemented. Applies `transform()` to each element, then computes an inclusive prefix sum. This is the correctness reference.
- **Version A** — `run_gpu_nostream()`: **you implement this**. Process the entire array on the GPU in a single batch using standard synchronous memory transfers.
- **Version B** — `run_gpu_streams()`: **you implement this**. Process the array in `N_CHUNKS` chunks using CUDA streams to overlap memory transfers with computation on the GPU.

The transformation applied to each element is done by the `transform()` function, followed by an inclusive prefix sum over the transformed array. Your both GPU versions must reproduce the same result as the CPU reference. The output of each functions is automatically evaluated in the `results_match()` function.

Task 1: Implement Version A and Version B. Implement both the `run_gpu_nostream()` and `run_gpu_streams()` bodies in `exercise1.cu`. The function signatures are shown in Listing 1. You must also write any CUDA kernels you need. The output of both functions must pass the `results_match()` check against the CPU reference. You are free to design the functions and the kernels, as you see fit.

Your code should compile and run with:

```
$ nvcc -O3 -arch=sm_75 -o exercise1_out exercise1.cu
$ ./exercise1_out
```

What to submit: The completed `exercise1.cu` with both functions and all kernels efficiently implemented, which compiles with the command above, runs and passes correctness checks.

Listing 1: Function signatures for Version A and Version B.

```
// Version A: process the full array synchronously, no streams.
void run_gpu_nostream(const float* h_in, uint* h_out);

// Version B: Use async memory transfers to overlap transfers with computation.
void run_gpu_streams(int n_streams, cudaStream_t* streams,
                    const float* h_in, uint* h_out);
```

Task 2: Running the Code Run your implementation with three values of `TRANSFORM_ITERS`: 1, 50, and 200. For each value, run the program at least 5 times and report the mean execution time. Fill in one copy of Table per configuration.

Questions to answer:

1. For each value of `TRANSFORM_ITERS`, which stream count gives the best performance? Does the optimal stream count change across the three configurations? Justify your answer.

2. Compare the time for Version B with 1 stream against Version A (no streams). Are they similar or different? If there is a difference, what do you think causes it?

What to submit:

- Three completed copies of the Table below, one per TRANSFORM_ITERS value, with mean times over 5 runs.
- Written answers to the two questions above, one short paragraph for each.

Configuration	Mean time (ms)	Speedup vs CPU
CPU reference	[]	–
Version A, no streams	[]	[]
Version B, 1 stream	[]	[]
Version B, 2 streams	[]	[]
Version B, 4 streams	[]	[]
Version B, 8 streams	[]	[]

Task 3: Profiling with nsys. Profile your implementation using the NVIDIA Nsight Systems:

```
$ nsys profile --stats=true -t cuda -o ex1_profile ./exercise1_out
```

From the nsys-ui timeline, extract the kernel time T_K , H→D transfer time T_{H2D} , and D→H transfer time T_{D2H} for each configuration. Focus only on the [All Streams] region in the profile. Use these to compute the **overlap index**:

$$I_{\text{overlap}} = 1 - \frac{T_{\text{wall}} - T_K}{T_{H2D} + T_{D2H}} \quad (1)$$

where:

- T_{wall} : total end-to-end wall time of the pipeline (ms). Consider that from the beginning of the first H2D to the end of the last D2H memory copy for that specific pipeline.
- T_K : total accumulated kernel execution time (ms)
- T_{H2D} : total accumulated Host-to-Device transfer time (ms)
- T_{D2H} : total accumulated Device-to-Host transfer time (ms)

All these values can be extracted from the profiler timelines. A value of $I_{\text{overlap}} = 1$ means transfers are fully hidden behind computation; $I_{\text{overlap}} = 0$ means fully sequential with no overlap; negative values indicate that the streaming overhead outweighs the benefit.

What to submit:

- **6 timeline screenshots** from nsys-ui, two groups of three at the same zoom level from the [All Streams] region, within each group:
 - TRANSFORM_ITERS = 1: one screenshot each for Version A (no streams), Version B with 1 stream, and Version B with your best stream count. The screenshots must show the **CUDA HW** rows so that memory transfers (H→D and D→H) and kernel execution bars are visible simultaneously.
 - TRANSFORM_ITERS=200: same three configurations, same requirement.
- The profiler output file named ex1_profile.nsys-rep only for the TRANSFORM_ITERS=200.

- Table below completed with values extracted from the profiler timelines for the same 6 configurations.
- A short written analysis describing what the timelines show: where overlap is present or absent, and where should or should not we use streams.

TRANSFORM_ITERS	Configuration	T_{wall} (ms)	T_K (ms)	T_{H2D} (ms)	T_{D2H} (ms)	I_{overlap}
1	no-stream	[]	[]	[]	[]	[]
1	1 stream	[]	[]	[]	[]	[]
1	best ($N = []$)	[]	[]	[]	[]	[]
200	no-stream	[]	[]	[]	[]	[]
200	1 stream	[]	[]	[]	[]	[]
200	best ($N = []$)	[]	[]	[]	[]	[]

Exercise 2 (8 [5+3] points): Memory Allocation and Transfer Benchmarking

You are given a benchmarking tool in `exercise2/mem_bench.cu` that tests how different Host and Device memory allocation strategies impact data transfer times and kernel execution over the PCIe bus. The host code, kernel, and timers are already implemented. The code is structured to automatically capture and report the times of the following pipeline stages using wall-clock timers and CUDA events:

- Allocations: Host and Device memory allocation times.
- Data Transfers: Host-to-Device (H2D) and Device-to-Host (D2H) transfer times.
- Execution: Kernel execution time.
- Deallocations: Host and Device memory freeing times.
- Overall Metrics: Total elapsed time and average time per full pipeline.

Your task is to benchmark these methods across a range of payload sizes to understand the trade-offs between allocation overhead, transfer times and bandwidths, and kernel execution.

Task 1: Benchmarking. Test all the combinations listed in table below, for each run, extract and record the median time reported for each phase of the pipeline.

Generate a series of line plots to visualize how the different memory strategies scale with payload size. For all plots, the X-axis must represent the Message Size (MB) (using a logarithmic scale), and the Y-axis must represent the Time (ms). Each valid host/device memory combination must be represented as a separate line with a distinct color and a corresponding label in the legend.

Generate the following plots:

1. H2D Transfer Time: Plot the median Host-to-Device transfer time across all message sizes.
2. Kernel Execution Time: Plot the median kernel execution time across all message sizes.
3. D2H Transfer Time: Plot the median Device-to-Host transfer time across all message sizes.
4. Host Allocation & Deallocation Time: For each run, calculate the sum of the median *host alloc* and *host free* times. Plot this combined host memory management overhead.
5. Device Allocation & Deallocation Time: Calculate the sum of the median *device alloc* and *device free* times. Plot this combined device memory management overhead.
6. Overall pipeline's average time.

Table 1: Benchmarking Experiment Parameters

Parameter	Values to Test
Message Sizes (<code>size_mb</code>)	1, 4, 16, 64, 256, 512
Host Memory (<code>host_type</code>)	0: Pageable 1: Pinned 2: Write-Combine 3: Managed 4: Mapped (Zero-Copy)
Device Memory (<code>dev_type</code>)	0: Standard 1: Managed

You can compile and run the code with:

```
$ nvcc -O3 -arch=sm_75 -o mem_bench mem_bench.cu
$ ./mem_bench <host_type> <device_type> <size_mb>
```

What to submit:

- The 6 requested plots in your report, with legends clearly identifying each host/device memory combination. Data sizes must be on the X-axes, properly set to a logarithmic scale, and the Y-axes must be time in milliseconds.
- Your raw outputs in a text file, together with the script you used to run the experiments.

Task 2: Analysis. Include a brief written analysis in your report answering the following based on your plots:

1. Which host memory allocation type is fastest for H2D transfers at large array sizes, and why?
2. Does the kernel execution time differ in different cases and which is faster? Justify your answer.
3. Is there a specific array size threshold where the "best" overall memory strategy changes? Explain why this shift occurs.

What to submit:

- Your written answers to the three questions above (1 short paragraph for each).

Exercise 3 (10 [4+3+3] points): Parallel Reduction

You are given a skeleton code in `exercise3/exercise3_reduce.cu` that computes the sum of an array of floating-point numbers. The code provides a CPU Reference (`cpu_reduce()`) which performs a sequential sum. Your task is to implement three different efficient GPU reduction strategies, as below:

- **Version A: Interleaved Tree** At each step of the reduction, pairs of elements separated by a growing stride are added together. In the first step, elements separated by a stride of 1 are added (e.g., index 0 + 1, index 2 + 3). In the next step, the stride doubles to 2 (e.g., index 0 + 2, index 4 + 6), and so on until the entire block is summed (See Figure 2, Version A).
- **Version B: Sequential Tree** This method halves the working set at each step. In the first step, the array is conceptually split in half, and the elements from the right half are added to the corresponding elements in the left half (stride = $N/2$). In the next step, the process repeats on the new, smaller left half (stride = $N/4$), continuing until only one element remains (See Figure 2, Version B).
- **Version C: Warp-Shuffle Primitives** This method performs a block reduction in two phases. First, each warp reduces its elements to a single value using CUDA warp-shuffle intrinsics¹ (`__shfl_down_sync`) for all intra-warp communication. Then, the per-warp results are combined using a second warp-shuffle reduction to produce the final block sum, written to the output via `atomicAdd`. Note that shared memory may be used to pass data between the two phases.

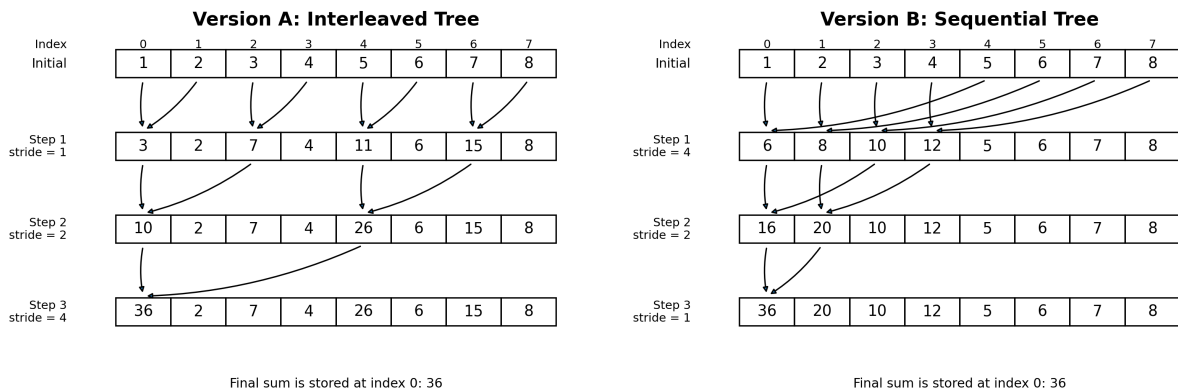
Reduction of Array [1, 2, 3, 4, 5, 6, 7, 8]

Figure 2: An example of the 2 reduction algorithms (Version A and B).

The code compiles and runs with the following commands. The function gets 2 parameters, the first one (`<method>`) gets from 0 to 3 (0=CPU, 1=Interleaved Tree, 2=Sequential Tree, 3=Warp-Shuffle Primitives) and the array size `<size_mb>` which gets a number in MB.

```
$ nvcc -O3 -arch=sm_75 -o ex3_reduce exercise3_reduce.cu
```

¹<https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#warp-shuffle-functions>

```
$ ./ex3_reduce <method> <size_mb>
```

Task 1: Implement Versions A, B, and C. Design the parallelization strategy for all three algorithms and implement the kernels and host functions for the functions: `reduce_interleaved()`, `reduce_sequential()`, and `reduce_warp()`. The timers are already provided in each function where they only time the kernel calls. You need to allocate/de-allocate the required memories, handle the data transfers and implement the required CUDA kernels for the requested algorithms.

What to submit: The completed `exercise3_reduce.cu` file with the three functions efficiently implemented, which compiles with the given command, runs and passes correctness checks. Explain your methodology of parallelizing the algorithms on the GPU in a short paragraph.

Task 2: Running the Code Across Input Sizes. Run each method over a range of input sizes (2, 16, 128, 1024, 4096) and compare their performance. Plot the execution times (in milliseconds) on the Y-axis against the array sizes (in MB) on the X-axis for all 4 methods (CPU, Version A, B and C) on a single figure.

What to submit: A plot, with the analysis of the achieved results in 1 short paragraph. A bash script to run all the experiments.

Task 3: Profiling and Bottleneck Analysis. Fix the input size at **4096 MB** and profile the three GPU versions (A, B, C) with `nsys`:

```
$ nsys profile --stats=true -t cuda -o ex3_profile
  ./ex3_reduce <method> <size_mb>
```

From the `nsys` profiling analysis, answer the following questions and support each answer with a screenshot from the profiler:

1. **Kernel duration.** Report the kernel execution time for each of the three GPU methods at 4096 MB, from the *Stats System View* of `nsys`. Which is fastest and by how much? What makes the difference in kernel times and what causes the methods times to be faster/slower?
2. **Memory transfer time.** Report the total memory transfer time and transferred size for each method obtained from the profiler's *stats view*.
3. Report the **GPU utilization** obtained from the profiler and write down the *Suggestions* provided by the profiler (find it on `nsys-ui` at the dropdown *Expert System View* -> *GPU Utilization*) for each of the kernels.

What to submit:

- Written answers to the questions above, each referencing a specific screenshot from the profiler where it shows the asked information.
- The three profiler output files (`*_profile.nsys-rep`) of the A, B, and C GPU versions, for the input size of 4096 MB.